

Функции и Классы в Python

Модуль 1: Основы функций.....	3
1.1 Чек-лист знаний.....	3
1.2 Глоссарий.....	3
1.3 Полезные ссылки.....	3
1.4 Задача на проверку.....	3
Модуль 2: Продвинутое функции и области видимости.....	5
2.1 Чек-лист знаний.....	5
2.2 Глоссарий.....	5
2.3 Полезные ссылки.....	5
2.4 Задача на проверку.....	5
Модуль 3: Введение в ООП и Классы.....	7
3.1 Чек-лист знаний.....	7
3.2 Справочная таблица основ класса.....	7
3.3 Полезные ссылки.....	7
3.4 Задача на проверку.....	8
Модуль 4: Наследование и Полиморфизм.....	9
4.1 Чек-лист знаний.....	9
4.2 Глоссарий.....	9
4.3 Полезные ссылки.....	9
4.4 Задача на проверку.....	9
Модуль 5: Инкапсуляция и Магические методы.....	11
5.1 Чек-лист знаний.....	11
5.2 Справочная таблица магических методов.....	11
5.3 Полезные ссылки.....	11
5.4 Задача на проверку.....	12
СБОРНИК ПРОВЕРОЧНЫХ ТЕСТОВ.....	13
Тест 1: Основы функций.....	13
Тест 3: Введение в ООП и Классы.....	17
Тест 4: Наследование и Полиморфизм.....	19
Тест 5: Инкапсуляция и Магические методы.....	21
ОТВЕТЫ И РАЗБОР ЗАДАНИЙ.....	23
Тест 1: Основы функций.....	23
Тест 2: Продвинутое функции и области видимости.....	24
Тест 3: Введение в ООП и Классы.....	25
Тест 4: Наследование и Полиморфизм.....	26
Тест 5: Инкапсуляция и Магические методы.....	27

Модуль 1: Основы функций

Что нужно уметь: Создавать переиспользуемые блоки кода, передавать данные внутрь функции и возвращать результат.

1.1 Чек-лист знаний

- [] Синтаксис объявления функции: ключевое слово `def`, имя, круглые скобки, двоеточие.
- [] Разница между `print()` (вывод на экран) и `return` (возврат значения из функции).
- [] Позиционные и именованные (keyword) аргументы.
- [] Аргументы по умолчанию (default arguments) и правило их расположения (в конце списка параметров).
- [] Аннотации типов (type hints) для аргументов и возвращаемого значения (`def func(a: int) -> int:`).

1.2 Глоссарий

- **Функция** – именованный блок кода, который выполняет определенную задачу и может быть вызван многократно.
- **Параметр** – переменная, указанная в скобках при *объявлении* функции.
- **Аргумент** – конкретное значение, передаваемое в функцию при её *вызове*.
- **Возвращаемое значение** – данные, которые функция отдает обратно в место своего вызова с помощью оператора `return`.

1.3 Полезные ссылки

- [Официальная документация: Определение функций](#)
- [PEP 484: Аннотации типов \(Type Hints\)](#)

1.4 Задача на проверку

"Генератор приветствий"

Напишите функцию `greet_user(name: str, role: str = "гость")`, которая возвращает строку формата: `"Здравствуйте, {name}! Ваша роль: {role}."`

Проверьте её работу, вызвав:

1. Только с именем (например, "Алексей").
2. С именем и ролью (например, "Мария", "администратор").

Модуль 2: Продвинутое функции и области видимости

Что нужно уметь: Работать с неопределенным количеством аргументов, использовать анонимные функции и понимать, где "живут" переменные.

2.1 Чек-лист знаний

- [] Распаковка аргументов: `*args` (кортеж позиционных аргументов) и `**kwargs` (словарь именованных аргументов).
- [] Лямбда-функции (`lambda`): синтаксис `lambda args: expression` и ограничения (только одно выражение).
- [] Правило областей видимости LEGB (Local, Enclosing, Global, Built-in).
- [] Ключевые слова `global` и `nonlocal` (и понимание, почему их лучше избегать без крайней необходимости).

2.2 Глоссарий

- **Лямбда-функция** – анонимная (безымянная) функция, записываемая в одну строку. Часто используется как аргумент для функций `map()`, `filter()`, `sorted()`.
- **Область видимости (Scope)** – контекст, в котором переменная существует и доступна для чтения/изменения.
- **Замыкание (Closure)** – функция, которая запоминает значения из своей внешней (enclosing) области видимости, даже когда внешняя функция уже завершила работу.

2.3 Полезные ссылки

- [Документация: Распаковка аргументов](#)
- [Документация: Лямбда-выражения](#)

2.4 Задача на проверку

"Умный фильтр"

Дан список чисел: `numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`.

Напишите функцию `filter_numbers(*nums, condition)`, где `condition` – это лямбда-функция. Функция должна возвращать новый список только тех чисел из `nums`, для которых `condition` возвращает `True`.

Пример вызова: `filter_numbers(*numbers, condition=lambda x: x % 2 == 0)` должен вернуть `[2, 4, 6, 8, 10]`.

Модуль 3: Введение в ООП и Классы

Что нужно уметь: Создавать собственные типы данных, объединять данные и поведение в одном объекте.

3.1 Чек-лист знаний

- [] Синтаксис создания класса: `class ClassName:` (CamelCase).
- [] Метод `__init__` (конструктор): инициализация атрибутов экземпляра.
- [] Параметр `self`: ссылка на текущий экземпляр класса (почему он всегда первый в методах).
- [] Создание экземпляра (объекта) класса и обращение к его атрибутам и методам через точку (`obj.attribute`).
- [] Разница между атрибутами экземпляра (`self.name`) и атрибутами класса (`ClassName.shared_var`).

3.2 Справочная таблица основ класса

Элемент	Что делает	Пример
<code>class Dog:</code>	Объявление класса	<code>class Dog:</code>
<code>def __init__(self, name):</code>	Инициализатор (конструктор)	<code>def __init__(self, name): self.name = name</code>
<code>self.attribute</code>	Атрибут экземпляра	<code>self.age = 0</code>
<code>def bark(self):</code>	Метод экземпляра	<code>def bark(self): print("Гав!")</code>

3.3 Полезные ссылки

- [Официальная документация: Классы в Python](#)

3.4 Задача на проверку

"Класс Книга"

Создайте класс `Book` с атрибутами `title`, `author` и `pages`.

1. В методе `__init__` добавьте атрибут `is_read = False`.
2. Напишите метод `read_book()`, который меняет `is_read` на `True` и выводит: `"Вы прочитали '{title}'"`.
3. Создайте экземпляр книги и вызовите этот метод.

Модуль 4: Наследование и Полиморфизм

Что нужно уметь: Создавать иерархии классов, переиспользовать код родительских классов и изменять поведение методов в дочерних классах.

4.1 Чек-лист знаний

- [] Синтаксис наследования: `class Child(Parent):`.
- [] Использование функции `super()` для вызова методов родительского класса (особенно `__init__`).
- [] Переопределение методов (Method Overriding): создание метода с тем же именем в дочернем классе.
- [] Полиморфизм: возможность вызывать один и тот же метод у разных объектов, реализующих его по-своему.
- [] Функции `isinstance(obj, Class)` и `issubclass(Child, Parent)`.

4.2 Глоссарий

- **Наследование** – механизм, позволяющий одному классу (дочернему) перенимать атрибуты и методы другого класса (родительского).
- **Переопределение (Overriding)** – замена реализации метода родительского класса в дочернем классе.
- **Полиморфизм** – свойство системы использовать объекты с одинаковым интерфейсом без информации о типе и внутренней структуре объекта.

4.3 Полезные ссылки

- [Документация: Наследование](#)

4.4 Задача на проверку

"Система сотрудников"

1. Создайте базовый класс `Employee` с атрибутами `name` и `salary`, и методом `get_bonus()`, который возвращает `salary * 0.1`.

2. Создайте дочерний класс `Manager`, который наследуется от `Employee`. В его `__init__` используйте `super()`.
3. Переопределите метод `get_bonus()` в `Manager` так, чтобы он возвращал `salary * 0.3`.
4. Создайте список из обычного сотрудника и менеджера, пройдите по нему циклом и выведите размер бонуса для каждого (демонстрация полиморфизма).

Модуль 5: Инкапсуляция и Магические методы

Что нужно уметь: Защищать внутренние данные класса от прямого изменения и настраивать встроенное поведение объектов (вывод на экран, сложение, длина и т.д.).

5.1 Чек-лист знаний

- [] Соглашения об инкапсуляции: `_protected` (защищенный, по соглашению) и `__private` (приватный, name mangling).
- [] Магические (dunder) методы: `__str__` (для пользователя) vs `__repr__` (для разработчика/отладки).
- [] Методы операторов: `__add__` (+), `__eq__` (==), `__len__` (len()).
- [] Понимание того, что в Python нет *настоящей* приватности, есть только соглашения и искажение имен (name mangling).

5.2 Справочная таблица магических методов

Метод	Когда вызывается	Пример реализации
<code>__str__(self)</code>	При вызове <code>print(obj)</code> или <code>str(obj)</code>	<code>return f"Объект: {self.name}"</code>
<code>__repr__(self)</code>	При выводе объекта в консоли/отладке	<code>return f"Class(name='{self.name}')</code>
<code>__len__(self)</code>	При вызове <code>len(obj)</code>	<code>return len(self.items)</code>
<code>__add__(self, other)</code>	При использовании оператора +	<code>return self.value + other.value</code>

5.3 Полезные ссылки

- [Документация: Модель данных \(Data Model\)](#)

5.4 Задача на проверку

"Умный кошелек"

Создайте класс `Wallet`.

1. В `__init__` создайте приватный атрибут `__balance` (начальный баланс передается как аргумент).
2. Напишите метод `deposit(amount)`, который пополняет баланс (проверьте, что `amount > 0`).
3. Реализуйте магический метод `__str__`, чтобы `print(wallet)` выводил: "Кошелек с балансом: {balance} руб."
4. Реализуйте метод `__add__`, чтобы можно было сложить два кошелька: `wallet3 = wallet1 + wallet2` (балансы складываются, возвращается новый объект `Wallet`).

СБОРНИК ПРОВЕРОЧНЫХ ТЕСТОВ

Тест 1: Основы функций

Часть А: Выберите один правильный ответ

1. Что вернет функция, если в ней нет оператора `return`?

- а) 0
- б) Пустую строку ""
- в) None
- г) Ошибку `SyntaxError`

2. Что выведет этот код?

```
def func(a, b=2):  
    return a + b  
  
print(func(3))
```

- а) 3
- б) 5
- в) Ошибку `TypeError`
- г) None

3. Какой из вариантов вызова функции `def test(x, y):` является некорректным?

- а) `test(1, 2)`
- б) `test(y=2, x=1)`
- в) `test(1, y=2)`
- г) `test(x=1, 2)`

4. Что произойдет при выполнении: `def my_func(l=[]):`
`l.append(1); return l`, если вызвать её дважды подряд без аргументов?

- а) Оба раза вернется `[1]`
- б) Первый раз `[1]`, второй раз `[1, 1]`
- в) Ошибка, так как списки нельзя использовать как аргументы по умолчанию
- г) Второй раз вернется `[2]`

5. Как правильно указать, что функция принимает целое число и возвращает строку?

- а) `def func(int a) -> str:`
- б) `def func(a: int) -> str:`
- в) `def func(a: int, return: str):`
- г) `def func(a) -> int -> str:`

Часть Б: Задания со свободным ответом (напишите код)

6. Напишите функцию `multiply`, которая принимает два числа и возвращает их произведение.

7. Перепишите вызов `func(10, 20, 30)` для функции `def func(a, b, c):`, используя именованные аргументы в порядке `c, a, b`.

8. Напишите лямбда-функцию, которая возводит число в квадрат.

9. Что выведет `print(type(lambda x: x))`?

10. Исправьте ошибку: `def greet(): return "Hi" print(greet())` (не хватает одного символа).

Тест 2: Продвинутое функции и области видимости

Часть А: Выберите один правильный ответ

1. Какой тип данных имеет переменная `args` внутри функции `def func(*args):?`

- а) Список (`list`)
- б) Кортеж (`tuple`)
- в) Словарь (`dict`)
- г) Множество (`set`)

2. Что выведет код?

```
x = 10

def change():
    x = 20

change()

print(x)
```

- а) 10
- б) 20
- в) None
- г) Ошибку `UnboundLocalError`

3. Как передать словарь `d = {"a": 1, "b": 2}` в функцию `def func(**kwargs):?`

- а) `func(d)`
- б) `func(*d)`
- в) `func(**d)`

г) `func(kwargs=d)`

4. Что вернет выражение `(lambda x, y: x if x > y else y)(5, 8)`?

а) 5

б) 8

в) True

г) Ошибку синтаксиса

5. Какое ключевое слово позволяет изменить переменную из внешней (не глобальной) области видимости внутри вложенной функции?

а) `global`

б) `outer`

в) `nonlocal`

г) `parent`

Часть Б: Задания со свободным ответом (напишите код)

6. Напишите функцию, которая принимает любое количество чисел и возвращает их сумму (используйте `*args`).

7. Дан список `words = ["apple", "banana", "kiwi"]`. Напишите одну строку кода с `sorted()` и `lambda`, чтобы отсортировать его по длине слов.

8. Напишите код, который демонстрирует использование `global` для изменения переменной `count = 0` внутри функции.

9. Что выведет `print(list(map(lambda x: x*2, [1, 2, 3])))`?

10. Можно ли в лямбда-функции использовать цикл `for`? (Ответьте Да/Нет и кратко почему).

Тест 3: Введение в ООП и Классы

Часть А: Выберите один правильный ответ

1. Как по соглашению (PEP 8) принято называть классы в Python?

- а) snake_case (my_class)
- б) CamelCase (MyClass)
- в) UPPERCASE (MY_CLASS)
- г) kebab-case (my-class)

2. Для чего нужен параметр `self` в методах класса?

- а) Это зарезервированное слово Python, его нельзя заменить
- б) Он передает ссылку на текущий экземпляр класса
- в) Он указывает на родительский класс
- г) Он нужен только для метода `__init__`

3. Что выведет этот код?

```
class Cat:
    legs = 4

c1 = Cat()
c2 = Cat()

c1.legs = 3

print(c2.legs)
```

- а) 3
- б) 4
- в) Ошибку `AttributeError`
- г) `None`

4. Как правильно создать экземпляр класса `User`?

- а) `u = User`
- б) `u = User()`
- в) `u = new User()`
- г) `u = create User`

5. Что произойдет, если забыть указать `self` первым параметром в методе экземпляра?

- а) Python добавит его автоматически
- б) Метод станет статическим
- в) При вызове метода возникнет ошибка `TypeError`
- г) Ничего, код будет работать

Часть Б: Задания со свободным ответом (напишите код)

6. Напишите пустой класс `Car` с использованием ключевого слова `pass`.

7. Добавьте в класс `Car` метод `__init__`, который принимает `color` и сохраняет его в атрибут экземпляра.

8. Как обратиться к методу `start_engine` у объекта `my_car`?

9. В чем разница между `MyClass.attr` и `self.attr`?

10. Напишите метод класса `@classmethod`, который просто выводит имя класса (используйте параметр `cls`).

Тест 4: Наследование и Полиморфизм

Часть А: Выберите один правильный ответ

1. Как вызвать конструктор родительского класса `Animal` внутри дочернего класса `Dog`?

- а) `Animal.__init__(self)`
- б) `super().__init__()`
- в) Оба варианта (а и б) верны
- г) Это происходит автоматически, вызывать не нужно

2. Что выведет код?

```
class A: pass

class B(A): pass

print(issubclass(B, A))
```

- а) True
- б) False
- в) Ошибку `TypeError`
- г) None

3. Если в дочернем классе определен метод с тем же именем, что и в родительском, какой метод будет вызван при обращении к объекту дочернего класса?

- а) Родительский
- б) Дочерний (произойдет переопределение)
- в) Вызовутся оба по очереди
- г) Ошибка `NameError`

4. Что проверяет функция `isinstance(obj, Class)`?

- а) Является ли `Class` наследником `obj`
- б) Является ли `obj` экземпляром `Class` или его наследником
- в) Равны ли `obj` и `Class` по значению
- г) Существует ли класс `Class`

5. Поддерживает ли Python множественное наследование (наследование от нескольких классов сразу)?

- а) Нет, только от одного
- б) Да, `class Child(Parent1, Parent2):`
- в) Да, но только для встроенных классов
- г) Только через интерфейсы

Часть Б: Задания со свободным ответом (напишите код)

- 6. Создайте класс `Bird`, наследуемый от класса `Animal`.
- 7. В классе `Bird` переопределите метод `make_sound()`, чтобы он возвращал "Чик-чирик".
- 8. Напишите код, который проверяет, является ли объект `sparrow` экземпляром класса `Bird`.
- 9. Что выведет `print(super().__class__)` внутри метода дочернего класса? (Кратко опишите суть).
- 10. Дан класс `Shape` с методом `area()`. Напишите класс `Circle`, который наследует `Shape` и реализует свой `area()` через `math.pi * r**2`.

Тест 5: Инкапсуляция и Магические методы

Часть А: Выберите один правильный ответ

1. Как в Python принято обозначать "защищенный" (protected) атрибут, который не стоит использовать вне класса?

- а) `__attr`
- б) `_attr`
- в) `attr_`
- г) `private attr`

2. Что произойдет при попытке обратиться к атрибуту `__secret` извне класса `Test`?

- а) Он вернет значение без проблем
- б) Он вернет `None`
- в) Возникнет ошибка `AttributeError` (из-за `name mangling`)
- г) Он автоматически станет публичным

3. Какой магический метод вызывается функцией `len(obj)`?

- а) `__size__(self)`
- б) `__count__(self)`
- в) `__len__(self)`
- г) `__length__(self)`

4. В чем главное отличие `__str__` от `__repr__`?

- а) `__str__` для отладки, `__repr__` для пользователя
- б) `__str__` должен быть читаемым для пользователя, `__repr__` должен быть однозначным (часто валидным кодом Python) для разработчика

в) Разницы нет, это синонимы

г) `__repr__` вызывается только при ошибке

5. Что вернет выражение `bool(obj)`, если в классе не определен метод `__bool__` или `__len__`?

а) Всегда False

б) Всегда True

в) None

г) Ошибку `TypeError`

Часть Б: Задания со свободным ответом (напишите код)

6. Напишите магический метод `__str__` для класса `Point(x, y)`, который возвращает строку `"Point(x, y)"`.

7. Как обратиться к приватному атрибуту `__data` класса `MyClass` извне, если это абсолютно необходимо (назовите искаженное имя)?

8. Реализуйте метод `__eq__` для класса `User`, чтобы два пользователя считались равными, если у них совпадает атрибут `user_id`.

9. Напишите метод `__add__` для класса `Vector`, который складывает координаты двух векторов и возвращает новый `Vector`.

10. Что выведет `print([1, 2] + [3, 4])` и какой магический метод списка за это отвечает?

ОТВЕТЫ И РАЗБОР ЗАДАНИЙ

Тест 1: Основы функций

1. в) None. Если `return` отсутствует или указан без значения, функция неявно возвращает `None`.
2. б) 5. Аргумент `b` имеет значение по умолчанию 2. $3 + 2 = 5$.
3. г) `test(x=1, 2)`. После именованного аргумента нельзя указывать позиционные.
4. **б) Первый раз `[1]`, второй раз `[1, 1]`. **Mutable (изменяемые) объекты в качестве аргументов по умолчанию создаются один раз при определении функции и сохраняют состояние между вызовами. Это частая ловушка!
5. б) `def func(a: int) -> str:`. Это стандартный синтаксис аннотаций типов (PEP 484).
6. `def multiply(a, b): return a * b`
7. `func(c=30, a=10, b=20)`
8. `lambda x: x ** 2`
9. `<class 'function'>`. Лямбда-функция – это всё равно функция.
10. Не хватает переноса строки или точки с запятой.
Правильно:

```
def greet():  
    return "Hi"  
  
print(greet())
```

Тест 2: Продвинутое функции и области видимости

1. б) Кортеж (tuple). `*args` собирает все лишние позиционные аргументы в кортеж.
2. а) 10. Присваивание `x = 20` внутри функции создает *новую локальную* переменную `x`, глобальная `x` не меняется (без ключевого слова `global`).
3. в) `func(**d)`. Оператор `**` распаковывает словарь в именованные аргументы.
4. б) 8. Лямбда возвращает `x`, если `x > y`, иначе `y`. Это аналог функции `max`.
5. в) `nonlocal`. Позволяет изменять переменные из внешней (но не глобальной) области видимости.
6. `def sum_all(*args): return sum(args)`
7. `sorted(words, key=lambda x: len(x))`
8. Код:

```
count = 0

def inc():

    global count

    count += 1
```

9. `[2, 4, 6].map` применяет лямбду к каждому элементу, `list` преобразует результат в список.
10. Нет. Лямбда-функция может содержать только одно *выражение* (expression), а не утверждение (statement) вроде цикла или присваивания.

Тест 3: Введение в ООП и Классы

1. б) CamelCase (MyClass). (PEP 8).
2. б) Он передает ссылку на текущий экземпляр класса. Это позволяет методу работать с данными конкретного объекта.
3. б) 4. `c1.legs = 3` создает атрибут экземпляра `legs` только для `c1`, перекрывая доступ к атрибуту класса. У `c2` атрибут экземпляра не создан, поэтому он обращается к атрибуту класса `Cat.legs`, который равен 4.
4. б) `u = User()`. Вызов класса с круглыми скобками создает его экземпляр.
5. в) При вызове метода возникнет ошибка `TypeError`. Python попытается передать экземпляр как первый аргумент, но метод не ожидает аргументов.

6. `class Car: pass`

7. Код:

```
class Car:

    def __init__(self, color):

        self.color = color
```

8. `my_car.start_engine()`

9. `MyClass.attr` обращается к атрибуту класса (общему для всех), `self.attr` обращается к атрибуту конкретного экземпляра.

10. Код:

```
class MyClass:

    @classmethod

    def print_name(cls):

        print(cls.__name__)
```


Тест 4: Наследование и Полиморфизм

1. в) Оба варианта верны. `super().__init__()` предпочтительнее и современнее, но `Animal.__init__(self)` тоже работает.
2. а) True. `B` является подклассом `A`.
3. б) Дочерний. Это и есть переопределение (overriding).
4. б) Является ли `obj` экземпляром `Class` или его наследником.
5. б) Да. Python поддерживает множественное наследование (в отличие от Java или C#).
6. `class Bird(Animal): pass`
7. Код:

```
class Bird(Animal):  
  
    def make_sound(self):  
  
        return "Чик-чирик"
```

8. `isinstance(sparrow, Bird)`

9. Это выведет класс, к которому принадлежит `self` (то есть дочерний класс), так как `super()` возвращает прокси-объект, делегирующий вызовы, но `__class__` всё равно указывает на исходный тип экземпляра.
10. Код:

```
import math  
  
class Circle(Shape):  
  
    def __init__(self, r):  
  
        self.r = r  
  
    def area(self):  
  
        return math.pi * self.r ** 2
```

Тест 5: Инкапсуляция и Магические методы

1. б) `__attr__`. Одно подчеркивание – это соглашение для разработчиков ("не трогать, это внутренняя кухня").
2. в) Возникнет ошибка `AttributeError`. Python применяет "искажение имени" (name mangling), превращая `__secret` в `_Test__secret`.
3. в) `__len__(self)`.
4. б) `__str__` для пользователя, `__repr__` для разработчика. `__repr__` должен быть максимально информативным и, в идеале, позволять воссоздать объект через `eval()`.
5. б) Всегда True. Если не определены ни `__bool__`, ни `__len__`, Python по умолчанию считает любой пользовательский объект истинным (True).
6. Код:

```
def __str__(self):  
    return f"Point({self.x}, {self.y})"
```

7. `obj.__MyClass__data` (это механизм name mangling, но использовать его в продакшене не рекомендуется)
8. Код:

```
def __eq__(self, other):  
    if isinstance(other, User):  
        return self.user_id == other.user_id  
    return False
```

9. Код:

```
def __add__(self, other):  
    return Vector(self.x + other.x, self.y + other.y)
```

10. Выведет `[1, 2, 3, 4]`. За это отвечает магический метод списка `__add__` (или `__iadd__` для `+=`).